

Prog2-nyilvántartás

Dokumentáció - Egyházi Zoltán Tamás

1. Specifikáció

1.1 A feladat leírása

A program egy konzolos egyetemi nyilvántartó rendszer, amely hallgatók és oktatók adatait kezeli. A program menürendszer segítségével teszi lehetővé az adatok listázását, szűrését, hozzáadását, szerkesztését és törlését.

1.2 Funkciók

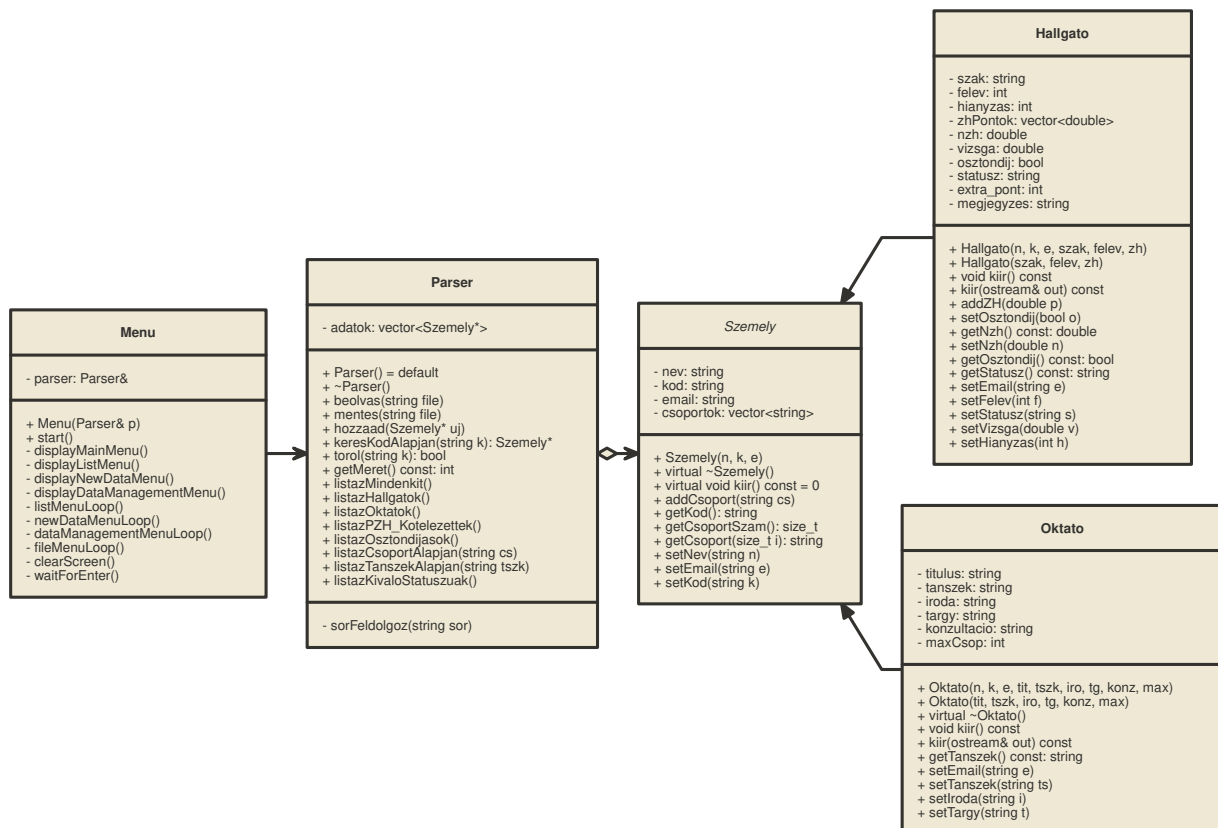
- **Listázás:** összes rekord, csak hallgatók, csak oktatók
- **Szűrés:** csoport, tanszék, osztóndíj, PZH-kötelezettség, státusz
- **Új adat felvétele:** hallgató vagy oktató hozzáadása
- **Adatok szerkesztése és törlése**
- **Fájl műveletek:** CSV fájlból olvasás és fájlba mentés

1.3 Technikai részletek

- A program CSV formátumú (Kulcs : Érték párok) fájlokat dolgoz fel, pontosvesszővel elválasztva.
 - A mezők sorrendje tetszőleges, az opcionális adatok hiánya nem okoz hibát.
 - A tesztek a `gtest_lite` keretrendszerrel készültek.
 - A memóriaszivárgás ellenőrzésére a `memtrace` szolgál.
-

2. Tervezés

2.1 UML osztálydiagram



2.2 Osztályok leírása

Szemely (Absztrakt őssosztály)

- **Közös attribútumok:** név, Neptun-kód, e-mail cím
- **Csoportkezelés:** `std::vector<std::string>` tárolja a csoporttagságokat, lehetővé téve, hogy egy személy tetszőleges számú csoporthoz tartozzon
- **Polimorfizmus:** virtuális destruktork és tiszta virtuális `kiir()` metódus, biztosítva a helyes memóriafelszabadítást és az egyedi adatközlést

Hallgato (Szemely leszármazott)

- **Tanulmányi adatok:** szak, aktuális félév, ösztöndíj státusz
- **ZH menedzsment:** `std::vector<double>` gyűjti a pontszámokat. A kulcs-értékpáros beolvasás révén tetszőleges számú részeredményt képes kezelni
- **Logika:** NZH pontszám, hiányzások figyelése, PZH-kötelezettség megállapítása

Oktato (Szemely leszármazott)

- Titulus, tanszék, iroda azonosító
- Tanított tárgy és konzultációs időpontok
- Maximálisan vállalható csoportok száma (maxCsop)

Parser (Adatkezelő motor)

- **Fájlkezelés:** CSV fájl betöltése és mentése
- **Dinamikus memóriakezelés:** `std::vector<Szemely*>` tárolja az objektumokat (nincs kézi `new[]/delete[]`)
- **Rugalmas feldolgozás:** kulcs-értékpárok alapján elemzi a bemenetet
- **Lekérdezések:** szűrési logikák (keresés kód alapján, ösztöndíjasok listázása, tanszéki statisztikák, stb.)

Menu (Vezérlő interfész)

- **Interakció:** konzolos megjelenítés, számozott menüpontok, felhasználói bevétel validálása
 - **Vezérlés:** a `start()` metódus ciklusban tartja a programot
 - **Adatfelvétel:** új rekordok adatainak bekérése, példányosítás, majd átadás a Parser-nek
-

3. Megvalósítás

3.1 Fordítás CMake segítségével

A projekt CMake (3.10+) építőrendszert használ. A `CMakeLists.txt` két külön futtatható állományt hoz létre:

- **program.exe** — a főprogram (menürendszer + adatkezelés). Forrásai: `main.cpp`, `Szemely.cpp`, `Hallgato.cpp`, `Oktato.cpp`, `Parser.cpp`, `Menu.cpp`, `memtrace.cpp`.
- **test.exe** — a tesztprogram (`gtest_lite` alapú egységtesztek). Forrásai: `test.cpp`, valamint a fenti osztályok forrásfájljai a `Menu.cpp` kivételével.

Fordítási flag-ek: `-Wall -Wextra -Werror -g`. A `-DMEMTRACE` makró globálisan definiált, így a `memtrace` könyvtár minden fordított egységben aktív.

3.2 Dinamikus memóriakezelés `std::vector` segítségével

A program három helyen használ dinamikus adattárolást, mindenhol a `std::vector` szabványos tárolóra építve. A kézi `new[]/delete[]` műveletek és a `Parser::atmeretez()` metódus teljesen kivezetésre kerültek.

Csoportok tárolása (Szemely::csoportok)

```
std::vector<std::string> csoportok;
```

Minden személyhez (hallgatóhoz és oktatóhoz egyaránt) tetszőleges számú csoport rendelhető. Az `addCsoport()` metódus a `push_back()` hívással fűzi hozzá az új csoportot a vektor végéhez:

```
void Szemely::addCsoport(std::string cs) {  
    csoportok.push_back(cs);  
}
```

ZH pontok tárolása (Hallgato::zhPontok)

```
std::vector<double> zhPontok;
```

A hallgatókhoz tartozó ZH pontszámokat a vektor a konstruktorban megadott méretben inicializálja nulla értékekkel:

```
Hallgato::Hallgato(... int zh)  
    : ... zhPontok(zh, 0.0) ...
```

Az `addZH()` metódus az első nulla értékű elem helyére írja be a pontszámot, ezzel megőrizve a rögzítés sorrendjét.

Személyek tárolása (Parser::adatok)

```
std::vector<Szemely*> adatok;
```

A `Parser` osztály egy `Szemely*` pointereket tároló vektorban tartja nyilván az összes hallgatót és oktatót. Az új elem hozzáadása egyszerű `push_back()` hívással történik:

```
void Parser::hozzaad(Szemely* uj) {  
    adatok.push_back(uj);  
}
```

A vektor automatikusan kezeli a memóriefoglalást és -felszabadítást, így nincs szükség kézi `new[]/delete[]` műveletekre. A `Parser` destruktora csak a tárolt objektumokat szabadítja fel (`delete`), a vektor magától felszabadul.

3.3 CSV fájl feldolgozása

A program a `Kulcs:Érték;Kulcs:Érték;...` formátumú CSV fájlokat dolgoz fel. Egy minta bemeneti sor:

```
Tipus:Hallgato;Nev:Kiss  
Anna;Kod:AABBCC;Szak:Mérnökinformatikus;Felev:2;ZH:15.5;ZH:18;Csoport:G01
```

A feldolgozás lépései:

1. **splitSor()** — a sort pontosvessző (;) mentén darabolja, majd minden darabot kettévág az első kettőspont (:) mentén, előállítva a (kulcs, érték) párok vektorát.
2. **getErtek()** — a párok vektorából kulcs alapján visszaadja a hozzá tartozó értéket.

3. **sorFeldolgoz()** — a **Típus** mező alapján eldönti, hogy **Hallgato** vagy **Oktato** objektumot kell-e létrehozni. A többi mezőt (**Név**, **Kod**, **Email**, **Szak**, **Csoport**, **ZH**, stb.) a megfelelő adattagokba tölti, majd a kész objektumot a **Parser**-hez adja.

Az adatokat a **beolvas()** metódus soronként olvassa be a fájlból, és minden sort átad a **sorFeldolgoz()** függvénynek. A **mentes()** metódus ennek fordítottját végzi: az objektumok **kiir(ostream&)** metódusával állítja elő a CSV formátumú kimenetet.

3.4 Polimorfizmus

Az osztályhierarchia középpontjában a **Szemely** absztrakt őssztály áll:

- **Szemely** — absztrakt őssztály, a közös adattagokat (név, kód, email, csoportok) és a **virtual void kiir() const = 0** tisztán virtuális metódust tartalmazza.
- **Hallgato** — **public Szemely** leszármazott, a hallgató-specifikus adatokkal (szak, félév, ZH pontok, osztöndíj, státusz).
- **Oktato** — **public Szemely** leszármazott, az oktató-specifikus adatokkal (titulus, tanszék, iroda, tantárgy).

A **Parser** az összes személyt **Szemely*** pointerként tárolja, így a virtuális **kiir()** metódus segítségével egységesen listázhatók:

```
void Parser::listazMindenkit() {
    for (size_t i = 0; i < adatok.size(); i++) {
        adatok[i]->kiir(); // polimorfikus hívás
    }
}
```

A típus-specifikus műveletekhez (pl. csak hallgatók listázása, osztöndíjasok szűrése, tanszék szerinti keresés) a **dynamic_cast** operátor biztosítja a futás idejű típusazonosítást:

```
void Parser::listazHallgatok() {
    for (size_t i = 0; i < adatok.size(); i++) {
        Hallgato* h = dynamic_cast<Hallgato*>(adatok[i]);
        if (h) h->kiir();
    }
}
```

Ez a megközelítés lehetővé teszi, hogy a rendszer bővíthető maradjon: új személytípus (pl. **Vendeg**) egyszerűen beilleszthető a **Szemely** osztályból származtatva, anélkül hogy a **Parser** vagy a **Menu** osztályokat módosítani kellene.

4. Tesztelés

4.1 Tesztkeretrendszer

A program egységtesztjei a **gtest_lite** keretrendszerrel készültek, amely a **JPorta** rendszerben előre elérhető. A tesztek külön fordítandó modulban (**test.cpp**) találhatók, és a **test.exe** futtatásával indíthatók.

A tesztesetek a program minden fontosabb funkcióját lefedik: konstruktorokat, getter/setter metódusokat, adatkezelő műveleteket és fájlból való beolvasást.

4.2 Tesztesetek

Összesen **37 teszteset**, mindegyik **SIKERES** eredménnyel.

Hallgato tesztek (11 eset)

Teszt	Leírás
KonstruktorAlap	Alap konstruktor szak, félév, ZH darabszám megadásával
KonstruktorTeljes	Teljes konstruktor névvel, kóddal, emaillel
AddZH	Egy ZH pontszám hozzáadása
AddMultipleZH	Több ZH pontszám egymás utáni hozzáadása
GetOsztondijAlapertelmezett	Alapértelmezett osztöndíj státusz (false)
SetOsztondijIgen	Osztöndíj beállítása true-ra
SetOsztondijNem	Osztöndíj beállítása false-ra
GetStatuszAlapertelmezett	Alapértelmezett státusz ("aktiv")
AddCsoport	Egy csoport hozzáadása
AddMultipleCsoport	Több csoport egymás utáni hozzáadása
GetCsoport	Csoport lekérése index alapján

Oktato tesztek (4 eset)

Teszt	Leírás
KonstruktorAlap	Alap konstruktor titulus, tanszék, iroda megadásával
KonstruktorTeljes	Teljes konstruktor névvel, kóddal, emaillel
GetTanszek	Tanszék nevének lekérése
AddCsoport	Csoport hozzáadása oktatóhoz

Parser tesztek (22 eset)

Teszt	Leírás
UresKonstruktor	Üres Parser létrehozása (0 elem)
HozzaadHallgato	Hallgató hozzáadása
HozzaadOktato	Oktató hozzáadása
HozzaadTobb	Több személy együttes hozzáadása
KeresKodAlapjanMegtalal	Keresés létező kóddal

Teszt	Leírás
KeresKodAlapjanNemTala l	Keresés nem létező kóddal
TorolHallgato	Hallgató törlése
TorolOktato	Oktató törlése
TorolNemLetezik	Törlés nem létező kóddal
TorolKettoUtanEgyMarad	Törlés után a másik elem megmarad
ListazMindenkit	Összes rekord listázása
ListazHallgatok	Csak hallgatók listázása
ListazOktatok	Csak oktatók listázása
ListazCsoportAlapjan	Szűrés csoport alapján
ListazCsoportAlapjanNemTala l	Szűrés nem létező csoportra
ListazTanszekAlapjan	Szűrés tanszék alapján
ListazTanszekAlapjanNemTala l	Szűrés nem létező tanszékre
ListazOsztondiJasok	Osztöndíjas hallgatók listázása
ListazKivaloStatuszuak	Kiváló státuszú hallgatók listázása
ListazPZH_Kotelezettek	PZH-ra kötelezettek listázása (NZH < 40)
BeolvasCsv	CSV fájl beolvasása (41 rekord)
BeolvasCsvOktatoEshallgato	CSV-ből oktatók és hallgatók szétválogatása

4.3 Memóriaszivárgás ellenőrzése

A program a **memtrace** könyvtárat használja a memóriaszivárgás detektálására. A **-DMEMTRACE** makró globálisan definiált, így minden fordítási egységben aktív. A **memtrace** az **atexit** regisztrációval automatikusan ellenőrzi a fel nem szabadított memóriaterületeket a program futásának végén.

A 37 tesztet futtatása után memóriaszivárgás nem keletkezik. Az összes dinamikusan foglalt objektum (Szemely leszármazottak, vektorok belső adatterületei) a Parser destruktorában és a `std::vector` RAII mechanizmusán keresztül felszabadul.

4.4 Eredmények összegzése

Tesztelt esetek: 37
 Sikeres: 37
 Hibás: 0
 Memóriaszivárgás: nincs

5. Felhasználói útmutató

5.1 Indítás

A program a `program.exe` futtatásával indítható:

```
./program.exe
```

Indításkor a program automatikusan betölti a `test_data.csv` fájlt. Amennyiben a fájl nem található, hibaüzenetet ír ki, és üres adatbázissal indul.

5.2 Főmenü

FŐMENÜ
1. Listázás 2. Új adat felvétele 3. Adatok szerkesztése / törlése 4. Fájl műveletek 0. Kilépés

A megfelelő szám billentyűzetről történő beírásával lehet navigálni.

5.3 Listázás almenü (1. pont)

LISTÁZÁS
1. Összes rekord 2. Csak hallgatók 3. Csak oktatók 4. Csoport alapján 5. Tanszék alapján 6. Ösztöndíjas hallgatók 7. PZH-ra kötelezettek 8. Kiváló státuszú hallgatók 0. Vissza

Lehetőség van a teljes adatbázis listázására, vagy szűrésre csoport, tanszék, ösztöndíj, PZH-kötelezettség és státusz alapján.

5.4 Új adat felvétele (2. pont)

A program bekéri az új személy adatait:

- **Hallgató:** név, Neptun kód (6 karakter), email, szak, félév (1-8), ZH pontszámok, csoportok
- **Oktató:** név, Neptun kód, email, titulus, tanszék, iroda, tantárgy, konzultációs idő, maximális csoportszám

5.5 Adatok szerkesztése / törlése (3. pont)

Neptun kód alapján lehet keresni, majd:

- Szerkeszteni az adatokat (név, email, szak, félév, stb.)
- Törölni a rekordot

5.6 Fájl műveletek (4. pont)

- Adatok mentése CSV fájlba
- Adatok betöltése CSV fájlból

6. Irodalom és eszközök

A projekt az alábbi eszközökkel és technológiákkal készült:

Eszköz	Verzió	Használat
C++	C++11	Programozási nyelv
CMake	3.10+	Építőrendszer, fordítási konfiguráció
gtest_lite	v4/2022	Egységtesztek keretrendszere
memtrace	—	Memóriaszivárgás detektálása
Doxygen	—	Forráskód dokumentáció generálása
GCC/Clang	—	Fordítás -Wall -Wextra -Werror -g kapcsolókkal
std::vector	STL	Dinamikus memóriakezelés
Git	—	Verziókövetés

Források:

- BME VIK Prog2 tárgy előadás- és laboranyagai
- A gtest_lite könnyűsúlyú tesztkeretrendszer (JPorta rendszerben)
- A memtrace memóriaszivárgás-ellenőrző könyvtár (Peregi Tamás, BME IIT)